

Workout Tracker API

A backend REST API for logging resistance training sessions, with derived analytics for one-rep-max estimation and personal-record detection.

1. Background and Context

Resistance training is fundamentally a data problem. Progress over weeks and months is invisible without records; existing consumer apps (Hevy, Strong, Jefit) handle the logging well but vary in how they surface derived metrics such as estimated one-rep max (1RM), volume by muscle group, and personal-record detection. This project is the design and implementation of a backend service that models a serious lifter's training workflow from first principles, with an emphasis on data integrity, derived analytics, and production-quality engineering practice.

The project is undertaken as an independent portfolio piece intended for inclusion on the developer's GitHub profile and CV. It is therefore expected to demonstrate not only functional correctness but also software-engineering maturity: testing, migrations, containerisation, deployment, and clear documentation.

2. Aims and Objectives

Primary aim. To design, implement, test, and deploy a REST API supporting the full lifecycle of resistance-training records — exercises, workouts, sets, and derived personal records — for a single authenticated user.

Learning objectives. On completion, the developer will have demonstrated:

- Competence in REST API design using FastAPI, including request validation, response modelling, and OpenAPI documentation.
- Relational data modelling with one-to-many and many-to-many relationships using SQLAlchemy 2.x and PostgreSQL.
- Schema migration management using Alembic.
- Authentication and authorisation using JSON Web Tokens (JWT) with password hashing.
- Automated testing using pytest, including fixtures, dependency overrides, and an isolated test database.
- Containerisation of the application and its database using Docker and Docker Compose.
- Deployment of the containerised application to a managed platform (Railway or Fly.io) with continuous integration via GitHub Actions.
- Production-quality project documentation, including a README, architecture diagrams, ER diagrams, and use-case documentation.

3. Functional Requirements

The system shall implement the following capabilities. Requirements are labelled **FR-n** for traceability against tests and commits.

ID	Area	Requirement
FR-1	Users	The system shall support user registration, login, and JWT-based authentication. Passwords shall be stored as bcrypt hashes.

ID	Area	Requirement
FR-2	Exercises	The system shall maintain a catalogue of system exercises that all authenticated users may read but not modify. Authenticated users shall additionally be able to create, read, update, and delete their own custom exercises. Each exercise (system or custom) shall have a name, one or more muscle-group associations (with a designated primary muscle group), and optional notes.
FR-2a	Seed catalogue	The system shall ship with a seed script that populates the database with an initial catalogue of common resistance exercises (minimum 30 entries spanning all major muscle groups). The seed script shall be idempotent and shall be invoked as part of the standard application setup.
FR-3	Workouts	Authenticated users shall be able to create, read, update, and delete workout sessions. Each workout shall have a date, optional name (e.g. "Push A"), and optional notes.
FR-4	Sets	Each workout shall contain one or more sets. Each set shall reference an exercise and shall record weight (kg), repetitions, optional RPE (1-10), and ordering within the workout.
FR-5	Listing and filtering	The workout listing endpoint shall support filtering by date range and pagination. The exercise listing endpoint shall support filtering by muscle group.
FR-6	1RM estimation	On retrieval of a set, the system shall return an estimated one-rep max calculated using the Epley formula: $1RM = \text{weight} \times (1 + \text{reps} / 30)$.
FR-7	Personal-record detection	On creation of a new set, the system shall determine whether the set constitutes a personal record (PR) for that exercise (by estimated 1RM) and shall record the PR if so.
FR-8	PR retrieval	Authenticated users shall be able to retrieve a list of their current personal records, one per exercise, ordered by date set.
FR-9	Authorisation scope	All write operations and all read operations on user-owned resources shall require authentication. A user shall not be able to read or modify another user's data. System exercises shall be readable by all authenticated users but shall not be writable by any user via the standard API.
FR-10	API documentation	The system shall expose interactive API documentation at /docs (Swagger UI) and /redoc (ReDoc), auto-generated from the FastAPI route definitions.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	All endpoints shall respond in under 200 ms for typical workloads (a single user with up to 1,000 workouts and 20,000 sets) on a local development environment.
NFR-2	Reliability	Database schema changes shall be managed exclusively through Alembic migrations. No direct schema modification shall occur outside of migration scripts.

ID	Category	Requirement
NFR-3	Security	Passwords shall be stored as bcrypt hashes with a cost factor of at least 12. JWTs shall have a configurable expiry, default 30 minutes. The application secret shall be loaded from environment variables, never committed to source control.
NFR-4	Test coverage	Automated tests shall achieve at least 70% line coverage of the application code, measured by coverage.py.
NFR-5	Reproducibility	The full application stack shall start with a single command (docker compose up) on any machine with Docker installed, with no manual configuration beyond a .env file.
NFR-6	Code quality	The codebase shall pass ruff linting and black formatting checks. These shall be enforced in CI.
NFR-7	Documentation	The repository shall include a README with project overview, setup instructions, architecture diagram, and a docs/ directory containing the ER diagram, use-case diagram, and API reference.

5. Out of Scope (v1)

The following features are explicitly excluded from version 1 of this project. They are recorded here to prevent scope expansion during development. Any of these may be addressed in a subsequent version, in a separate repository, or never. They shall not appear in the v1 codebase, not even as commented-out code or TODO markers.

- Frontend web application (the API is consumed via Swagger UI and HTTP clients in v1).
- Mobile application of any kind.
- Social features: friends, sharing, public profiles, leaderboards.
- Cardiovascular exercise tracking (running, cycling, swimming). v1 is resistance training only.
- Nutritional tracking, macros, calorie counting.
- Body-measurement tracking (weight, body fat, circumference measurements).
- Photo or video attachments on sets or workouts.
- Integration with wearables, smart-watches, or third-party fitness platforms.
- AI-generated workout recommendations or programme generation.
- Multi-tenancy beyond per-user data isolation (no teams, gyms, or coaches).

6. Technology Stack

Layer	Technology	Version
Language	Python	3.12
Web framework	FastAPI	0.115.x
ASGI server	uvicorn	0.32.x
ORM	SQLAlchemy	2.0.x
Migrations	Alembic	1.13.x
Database	PostgreSQL	16
Validation	Pydantic	2.x (bundled with FastAPI)

Layer	Technology	Version
Authentication	python-jose (JWT) + passlib[bcrypt]	latest
Testing	pytest + pytest-asyncio + httpx	latest
Coverage	coverage.py	latest
Linting	ruff	latest
Formatting	black	latest
Containerisation	Docker + Docker Compose	latest
CI	GitHub Actions	n/a
Deployment	Railway (primary) or Fly.io (fallback)	n/a

7. Deliverables

The completed project shall consist of the following artefacts, all hosted in a single public GitHub repository:

- **Source code** — a working FastAPI application implementing all functional and non-functional requirements above.
- **Automated test suite** — pytest-based, meeting the coverage threshold in NFR-4.
- **Alembic migration scripts** — incremental, in chronological order, reproducing the schema from empty database to current state.
- **Seed-data script** — an idempotent script (invoked via a Makefile target or CLI command) that populates the system exercise catalogue, per FR-2a.
- **Docker artefacts** — Dockerfile for the application, docker-compose.yml for the full stack.
- **CI configuration** — .github/workflows/ci.yml running lint, format check, tests, and coverage on every push.
- **Documentation** — README.md (overview, setup, usage, architecture), docs/ER.png, docs/use-case.png, docs/architecture.md (Mermaid).
- **Deployed instance** — a publicly accessible URL with Swagger UI live, linked from the README.
- **This brief** — included in the repository as docs/brief.pdf for context.

8. Required Design Artefacts (pre-implementation)

Before any application code is written, the following design artefacts shall be produced and committed to the repository under docs/. They form the design phase of the project and are themselves part of the assessed deliverable.

- **Entity-Relationship Diagram** — drawn in dbdiagram.io, exported as PNG and DSL source. Shows all entities, attributes, primary keys, foreign keys, and cardinalities.
- **Use Case Diagram** — UML use-case diagram drawn in draw.io, exported as PNG and .drawio source. Shows the single actor (User) and all use cases corresponding to FR-1 through FR-10.
- **Architecture Diagram** — Mermaid diagram embedded in docs/architecture.md, showing client, FastAPI application, SQLAlchemy layer, PostgreSQL, Docker Compose boundary, and external CI/deployment.
- **API Endpoint Specification** — a markdown table in docs/api.md listing every endpoint, HTTP method, path, request body schema, response schema, and authorisation requirement.

9. Milestones

#	Milestone	Definition of done
M0	Brief signed off	This document committed to repo at docs/brief.pdf.
M1	Design phase complete	All four artefacts in Section 8 committed to docs/.
M2	Skeleton + first endpoint	FastAPI runs, /health endpoint returns 200, Swagger UI loads.
M3	Database + first model	Postgres in Docker, Alembic configured, User and Exercise tables created.
M4	Authentication	Register, login, JWT-protected endpoints working. FR-1, FR-9 met.
M5	Exercises CRUD	FR-2 met. Tested.
M6	Workouts and sets CRUD	FR-3, FR-4, FR-5 met. Tested.
M7	Derived analytics	FR-6, FR-7, FR-8 met. Tested.
M8	Production hygiene	Coverage at 70%, ruff/black clean, CI green, fully Dockerised.
M9	Deployed	Live URL in README, Swagger UI accessible publicly.

10. Success Criteria

The project will be considered complete when every item in the list below is true. These criteria are deliberately binary; partial credit is not awarded.

- All functional requirements FR-1 to FR-10 are implemented and verified by automated tests.
- All non-functional requirements NFR-1 to NFR-7 are met.
- All design artefacts in Section 8 are committed to the repository.
- All deliverables in Section 7 are present in the repository.
- The deployed instance is reachable and Swagger UI loads.
- The CI pipeline is green on the main branch.
- Nothing from Section 5 (Out of Scope) appears in the codebase.

11. Acknowledgement

By beginning implementation, the developer acknowledges that this brief defines the scope of v1 and that any change to scope shall be made by amending this document and incrementing its version number, not by ad-hoc decisions during implementation. The Out of Scope list in Section 5 is binding for v1.

Developer: Uluç Terek

Project: Workout Tracker API

Brief version: 1.1

Status: Active

12. Changelog

Version	Date	Change
1.0	24 May 2026	Initial brief.

Version	Date	Change
1.1	24 May 2026	Adopted hybrid exercise-catalogue model. FR-2 reworded: users may read system exercises and CRUD their own custom exercises. Added FR-2a for seed catalogue. Updated FR-9 to reflect the new tri-tier authorisation (user-owned, other-user, system). Added seed-data script to Section 7 deliverables. Rationale: aligns the data model with how production fitness apps actually work (Hevy, Strong, Jefit) and avoids retrofitting later. Decision made during design phase, before any code written.